



Hierarchy aware-based multi-task learning for user location prediction

Yahui Wang^{1,2} · Hongchang Chen² · Shuxin Liu² · Junjie Zhang² · Lan Wu² · Xiaoyan Cui³ · Yuxiang Hu²

Accepted: 30 June 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

With advancements in high-precision positioning technology and the availability of commercial map services, LBSNs can now acquire both geographic coordinates and precise category information of locations. However, most existing models for user location prediction suffer from two significant limitations: neglecting category information and overlooking the hierarchical relationship between specific locations and their corresponding categories. To overcome these challenges, we propose the hierarchy aware-based multi-task learning (HAMTL) framework, which jointly predicts the next location and its category. HAMTL extracts hierarchy information by constructing a hierarchy tree, which is then fused with other feature information before encoding. Finally, a hierarchical decoder is designed for prediction. Extensive experiments conducted on two real-world datasets demonstrate that HAMTL outperforms seven baselines across all evaluation metrics, achieving a maximum *Acc@1* improvement of over 18% compared with the state-of-the-art method. In addition, we show that the hierarchical decoder provides advantages over HAMTL on the basis of the effective modeling of hierarchy information.

Keywords Multi-task learning · Location prediction · Hierarchy information · Hierarchical decoder · Heterogeneous graph neural network

1 Introduction

With the widespread use of smart mobile devices, location-based social networks (LBSNs), such as Foursquare, Twitter, WeChat, and TikTok, have become essential to people's daily lives [1, 2]. Mining user mobility patterns from LBSN data and predicting their future visit locations can have significant impacts on various areas in modern society, such as urban planning [3], commercial recommendations [4],

Extended author information available on the last page of the article

Published online: 29 July 2025

Springer

and public safety [5]. Therefore, location prediction via LBSN data has emerged as a research hotspot in both industry and academia.

The proliferation of deep learning technologies has spurred the development of advanced algorithms capable of predicting users' next locations with high accuracy. Initially, these models utilized recurrent neural networks (RNNs) to analyze temporal patterns in user mobility data, aiming to accurately forecast their next movements [6, 7]. While RNNs are well-suited for processing sequential data, their effectiveness wanes with longer sequences because of their limited ability to maintain long-term dependencies. To mitigate these shortcomings, researchers have shifted towards the use of attention mechanisms, which enhance model performance by focusing on critical segments of data sequences, thus improving the accuracy of predictions [8, 9]. However, traditional models have largely focused on individual user data, overlooking the rich insights that could be derived from analyzing interactions among multiple users. To address this gap, recent advancements have incorporated graph neural networks (GNNs) into prediction frameworks [10–12]. By leveraging the networked interactions among users, GNNs enable a more detailed exploration of the spatial connections between locations. Moreover, the integration of self-attention with GNNs has significantly deepened the analysis of spatial-temporal relationships, leading to notable enhancements in prediction accuracy. Further refinements in the field have involved integrating location semantics into these models, providing enriched contextual insights that improve the understanding of user behavior [13, 14]. These advancements represent a significant leap forward from earlier approaches, offering more precise and contextually aware predictions by integrating a broader spectrum of data inputs.

Despite their impressive performance, these models still face *two* major challenges. *First*, they often fail to incorporate the category information associated with each location. Modern LBSNs not only provide precise coordinates but also include category information, which can reflect the semantic context of user activities. For example, location A, with coordinates (34.795346, 113.679853), is categorized as a hotspot restaurant, suggesting that people visit this location primarily for dining. This leads to a key research question: how can category information be utilized to enhance location prediction performance? *Second*, locations and their corresponding categories exhibit hierarchical relationships at different levels. For example, if location A mentioned above is a Chinese restaurant labeled as 1 (as shown in Fig. 1), in addition to the specific category of the hotspot, A also belongs to more abstract categories, such as Chinese restaurant and restaurant. These categories form an inherent hierarchy similar to a hierarchy tree with a specific location. This raises another interesting research question: how can we effectively model this intrinsic hierarchy information?

In this study, we present a novel hierarchy aware-based multi-task learning (HAMTL¹) framework to address the limitations of the above methods. Our framework consists of two tasks: location prediction as the main task and category prediction as the auxiliary task. The auxiliary task improves the

¹ <https://github.com/mrc-wyh/HAMTL>.

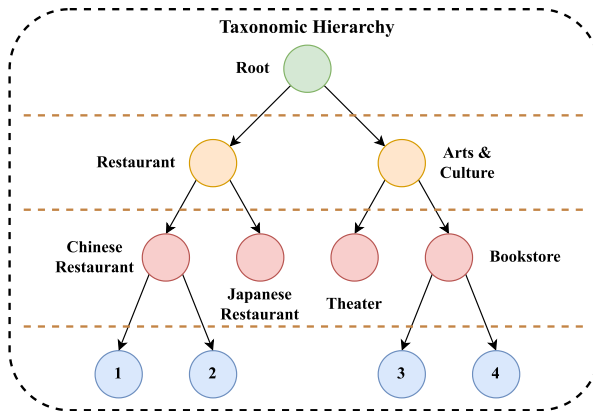


Fig. 1 Hierarchy tree. The figure illustrates three levels, which may vary in practice and may be more or less than three levels

performance of the main task by exploiting the hierarchical relationship between locations and categories. To represent this relationship, we construct a hierarchy tree, as illustrated in Fig. 1. We then extract top-to-bottom paths and concatenate the embeddings of all nodes on these paths to create the hierarchy feature. We employ a heterogeneous GNN to learn the location embeddings and propose three methods to obtain category embeddings within the hierarchy tree. Furthermore, we incorporate temporal and user features into our model and obtain user trajectory embeddings. On the basis of the Transformer encoder [15], which has demonstrated excellent performance in time-series tasks, we encode user trajectory embeddings into a latent representation. We then design a hierarchical decoder that first predicts the category and then refines the location based on the predicted category. Through extensive experiments conducted on two real-world datasets, we demonstrate that HAMTL significantly outperforms existing state-of-the-art methods.

The main contributions of this study can be summarized as follows:

- We introduce a novel multi-task learning framework, called HAMTL, which employs category prediction as an auxiliary task to enhance the primary task of location prediction. Experimental results demonstrate that HAMTL consistently surpasses state-of-the-art methods.
- To the best of our knowledge, we are among the first to introduce a hierarchy tree to model the inherent hierarchical relationships between locations and their corresponding categories. Moreover, we propose three methods to obtain category embeddings within the hierarchy tree and compare their impacts on model performance.
- We propose a hierarchical decoder that first predicts the category and then refines the location based on the predicted category, taking into account prior hierarchy information.

- We compare the impact of hierarchical and parallel decoder architectures on model performance and find that the hierarchical architecture provides more benefits on the basis of the effective modeling of hierarchy information.

The remainder of this paper is structured as follows: Sect. 2 reviews related work on location prediction and multi-task learning. Section 3 defines the problem of location prediction. Section 4 describes our proposed method in detail. Section 5 presents and discusses the experimental results. Lastly, Sect. 6 concludes the paper and outlines potential future research directions.

2 Related work

2.1 Location prediction

Location prediction plays a crucial role in enhancing a variety of location-based services, including public safety initiatives, business recommendations, and traffic management strategies. A diverse array of methods have been applied to refine this technology, with RNNs at the forefront [6, 7, 16–18]. For example, the ST-RNN model [16] integrates both spatial and temporal dimensions with traditional RNN frameworks, significantly increasing the prediction accuracy. Similarly, the LSTPM model [7] combines insights into users' long-term and short-term preferences with the spatial dynamics of their recent movements, thereby improving predictions of user mobility.

Recent developments in GNNs [19–21] and the Transformer architecture [15] have led to groundbreaking improvements across various domains. This evolution has spurred a shift towards merging these two powerful technologies to enhance the accuracy of location prediction models [10, 11, 13, 22–27]. For instance, GETNext [10] employs a hybrid approach that combines a trajectory flow map with a Transformer framework and captures both spatial and temporal relationships among locations. Similarly, TGSTAN [11] utilizes a dynamic trajectory graph combined with a spatial-temporal attention network, effectively learning spatial correlations and temporal dynamics. This model adapts dynamically to real-time data and collective user behavior, thus maintaining accuracy over varying conditions. KGNext [26] integrates a knowledge graph with a Transformer to address common and user-specific preferences in point-of-interest (POI) recommendations simultaneously. This model leverages a transitional-interactive knowledge graph to develop deep insights into user interactions and transitions, which are then processed through graph embeddings and the Transformer framework. This method adapts dynamically to both prevailing trends and individual user preferences, effectively managing uncertainties in location data to deliver precise predictions.

2.2 Multi-task learning

Multi-task Learning (MTL) is a machine learning paradigm that enables a model to learn multiple related tasks simultaneously, enhancing performance and

generalization across tasks [28, 29]. MTL is broadly classified into parallel and cascaded types. In this work, we focus on cascaded MTL, where the computations for subsequent tasks depend on the outputs of preceding ones, exploiting the inherent data structure to increase model efficiency and effectiveness.

Cascaded MTL has shown remarkable efficacy in areas such as natural language processing [30, 31] and computer vision [32, 33]. For example, Triple-MRC [31] introduces a cascaded learning model that manages interconnected subtasks such as aspect term extraction, opinion extraction, and sentiment classification. These tasks are sequentially dependent, effectively using outputs of earlier tasks to enhance the performance of subsequent tasks. Similarly, this work [32] proposes a multi-scale cascaded multi-task network for simultaneously segmenting and classifying brain tumors in magnetic resonance imaging (MRI) images. The cascaded approach within the network allows the use of output maps from earlier stages to refine and enhance segmentation and classification in later stages. In the realm of user multi-behavior recommendation, cascaded task relationships are particularly impactful [34–38]. For example, MB-CGCN [35] processes different user behaviors, such as clicks, carts, and purchases, in a sequential manner. It uses the embeddings from one behavior to inform the next, acknowledging and leveraging interdependencies among behaviors to increase predictive accuracy. Furthermore, CEMBR [38] tackles the challenges of heterogeneous and multi-type behaviors in recommendation systems via a cascaded structure that allows information from one behavior to enhance the learning of subsequent behaviors. This method shows how cascaded architectures can effectively manage complex, multi-behavior user data to refine recommendations.

Motivated by the aforementioned studies, we propose a novel hierarchical multi-task learning framework that leverages the extraction of hierarchy relationships between locations and their corresponding categories.

3 Problem description

Let $U = \{u_1, u_2, \dots, u_{|U|}\}$ be the user set and let $L = \{l_1, l_2, \dots, l_{|L|}\}$ be the location set. Each location $l_i \in L$ is represented by a tuple $\langle i, lat, lon, cat \rangle$, where i is the location ID, lat is the latitude, lon is the longitude, and cat is the location category.

Definition 1 (Spatial-Temporal Point) A spatial-temporal point is a pair of $p = \langle l, t \rangle, l \in L$, indicating that a user visited location l at time t .

Definition 2 (Trajectory) The i_{th} trajectory of a user u in U is denoted as $S_u^i = \{p_{u,i}^1, p_{u,i}^2, \dots, p_{u,i}^s\}$, where $p_{u,i}^j$ represents the j_{th} spatial-temporal point visited by the user within that trajectory. These spatial-temporal points are arranged chronologically, mapping the user's movement across various locations over time. The number of spatial-temporal points can differ among trajectories. Due to the intermittent nature of user location visits, a user's visit trajectory is typically segmented into multiple trajectories by applying a specific time interval threshold. Consequently, the set of trajectories for user u is denoted by $T_u = \{S_u^1, S_u^2, \dots, S_u^m\}, u \in U$.

Definition 3 (*User location prediction*) Let $T_u, u \in U$ be the historical trajectories and let $\tilde{S}_u = \{p_u^1, p_u^2, \dots, p_u^r\}, u \in U$ be the current trajectory of user u , respectively. User location prediction is to infer p_u^{r+1} in $\tilde{S}_u$ based on T_u .

4 Our proposed method

4.1 Overview

Figure 2 illustrates the HAMTL pipeline. We construct a hierarchy tree from the specific locations and their categories, and extract paths as hierarchy information. We then generate the trajectory embedding by concatenating the hierarchy information with the user embedding and time embedding, and use a Transformer-based encoder to obtain the final representations. Finally, we design a novel hierarchical decoder that incorporates prior hierarchy information, i.e., it first predicts the location category, and then predicts the specific location based on the category and prior hierarchy information.

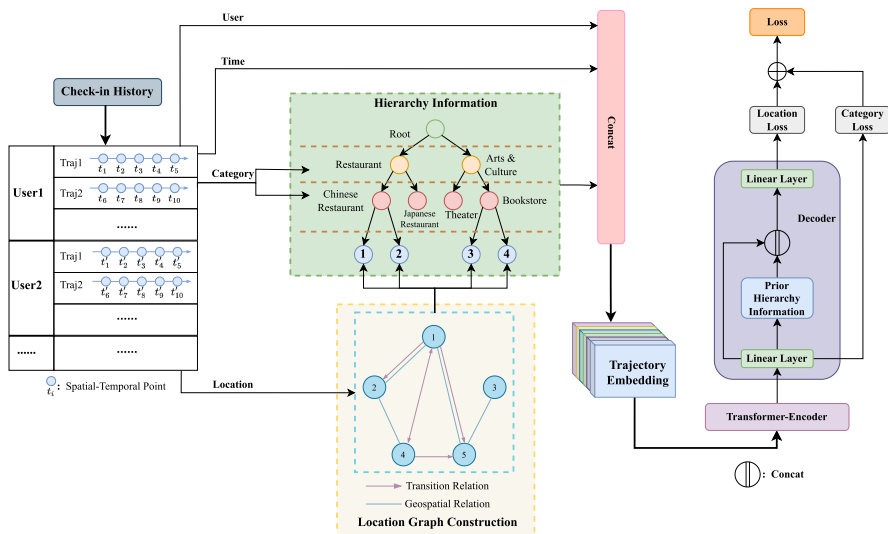


Fig. 2 The HAMTL pipeline consists of four main steps. First, user trajectories are extracted from historical check-in data as part of the data preprocessing phase. Next, user trajectory embeddings are generated, which include user, time, and hierarchy information embeddings. These hierarchy information embeddings are obtained by deriving location and category embeddings using a hierarchy tree. In particular, the location embeddings are obtained by a heterogeneous GNN applied to a constructed location graph. Next, the user trajectory embeddings are encoded into a latent representation utilizing a Transformer-based encoder. Finally, the category and location prediction is performed through a hierarchical decoder that sequentially predicts the category followed by refining the location based on the predicted category

4.2 Hierarchy information extraction

To extract hierarchy information, we first construct a hierarchy tree (see the green background area in the middle of Fig. 2) from the specific locations and their categories, then obtain the top-to-bottom paths, and finally concatenate the embeddings of all nodes on the paths. This method ensures that locations within the same category are more similar to each other in the embedding space. Therefore, our next step is to generate the embeddings for the specific locations and their categories.

4.2.1 Location embedding generation

In this study, we built a location graph based on the transition and geospatial relationships among locations in the trajectories of all users, and apply a GNN to learn the location embedding from this graph. The transition relationship reflects the frequency of movement from one location to another, and the geospatial relationship reflects the distance between locations.

Location graph construction Previous research indicates that human mobility patterns are regular and predictable, with individuals often returning to a few preferred locations [39]. Moreover, the visit histories of users can indicate their common destinations, and both the frequency and timing of movements are also relevant factors. Location transitions reflect the mobility patterns of users among locations. Therefore, it is crucial to consider the global location transition of all users to effectively model and forecast user movements. In our previous work [13], user check-ins are organized into distinct trajectories by a 24-hr interval (shown in the table on the left side of Fig. 2). For each trajectory, a weighted and directed edge connects consecutive check-in locations. The weight of the edge, ω_{ij} , from location l_i to l_j , is calculated using the equation:

$$\omega_{ij} = \frac{f_{ij}}{\sum_{l_k \in L} f_{kj}} \quad (1)$$

where f_{ij} is the transition frequency from l_i to l_j .

Tobler's first law of geography posits that while everything is related to everything else, nearer entities share a stronger connection than those further apart [40]. Supporting this, Gonzalez et al. [39] demonstrated that individuals typically maintain a consistent travel distance over time. These insights suggest a substantial impact of geospatial proximity on personal mobility patterns. In this study, locations that are within 1000 m of each other are connected by undirected and unweighted edges, indicating their spatial closeness.

On the basis of the above relationships, we construct a heterogeneous location graph, $\mathcal{G}$, where nodes represent specific locations and edges reflect the varied relationships between them. To derive location embeddings from $\mathcal{G}$, we utilize a GNN

method, which involves two main stages. First, we generate temporary embeddings for each location based on the types of edges connecting them. Second, these embeddings are integrated into updated location embeddings using a semantic attention mechanism. This approach ensures that the embeddings accurately capture the complexity of the relationships in the graph.

Location graph learning Updating the node features on transition edges resembles the method used in Graph Attention Networks (GAT) [41], where different weights are assigned to neighbors during a node's update. However, unlike GAT, which automatically learns these weights, our approach derives them statistically from the training data, as detailed in Eq. (1). The process for updating the node features on transition edges is as follows:

$$h_t^{(l)} = \sum_{i \in \mathcal{N}_j} \omega_{ij} h_p^{i(l-1)}, l = 1, 2, \dots \quad (2)$$

where ω_{ij} represents the weight of the edge from node i to j , $\mathcal{N}_j$ is the neighbor set of node j , $h_p \in \mathbb{R}^{D_1}$ denotes the node embeddings, $h_t \in \mathbb{R}^{D_1}$ indicates the updated node embeddings based on transition edges, and l is the number of network layers. When l is 1, $h_p^{(0)}$ represents the initial location embeddings, which is generated by embedding the location ID.

The node feature update process for geospatial edges closely aligns with the methodology of Graph Convolutional Networks (GCN) [19], which involves averaging the features of neighboring nodes during an update. The update formula is expressed as:

$$h_g^{(l)} = \sigma(\hat{A}_g h_p^{(l-1)} W_g^{(l-1)}) \quad (3)$$

where $\hat{A}_g$ represents the normalized adjacency matrix for geospatial edges, W_g denotes the feature transformation matrix to be learned, $\sigma(\cdot)$ is the activation function, and h_g is the updated node embeddings based on geospatial edges.

To streamline computation and prevent overfitting, we adopt the methodologies of SGC [42] and LightGCN [20], which eliminate the feature transformation matrix and activation function. As a result, Eq. (3) is simplified to:

$$h_g^{(l)} = \hat{A}_g h_p^{(l-1)}, \quad l = 1, 2, \dots \quad (4)$$

This revision facilitates more efficient node feature updates on geospatial edges.

For the temporary location embeddings, h_t and h_g , we employ semantic attention to determine their importance and allocate weights accordingly when fusing the two types of node features. The feature fusion process, utilizing semantic attention, is described by the following equations:

$$\begin{aligned}
h_p^{(l)} &= \sum_{e \in \{g, t\}} \beta_e^{(l)} h_e^{(l)} \\
Sc_e^{(l)} &= \frac{1}{|L|} \sum_{i \in L} \mathbf{q}^T \cdot \tanh(\mathbf{W} \cdot h_e^{i(l)} + b), \quad e \in \{g, t\} \\
\beta_g^{(l)} &= \frac{\exp(Sc_g^{(l)})}{\sum_{e \in \{g, t\}} \exp(Sc_e^{(l)})} \\
\beta_t^{(l)} &= \frac{\exp(Sc_t^{(l)})}{\sum_{e \in \{g, t\}} \exp(Sc_e^{(l)})}
\end{aligned} \tag{5}$$

where g and t represent the geospatial and transition edges, respectively, β_e represents the weight of node features based on edge type e , $\mathbf{W} \in \mathbb{R}^{D_1 \times K}$ is the feature transformation matrix, and $\mathbf{q} \in \mathbb{R}^{K \times 1}$ is the semantic attention vector. $\mathbf{W}$ and $\mathbf{q}$ are learnable parameters.

Equations (2)–(5) outline the node feature updating process for a single layer (i.e., from the $(l-1)_{th}$ to the l_{th} layer) in a heterogeneous GNN applied to the location graph. To enhance learning performance, multiple layers may be stacked.

4.2.2 Category embedding generation

There are several ways to generate category embeddings, such as the absolute position encoding method used in Transformer, the learnable method used in BERT [43], and the generative approach, where the hierarchy tree is transformed into a heterogeneous graph with two types of nodes: category nodes and location nodes. In this approach, the category embeddings are derived from the embeddings of the location nodes on the heterogeneous graph.

Absolute position encoding method This method follows the positional encoding technique in Transformer, which is defined by

$$\begin{aligned}
PE(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \\
PE(pos, 2i+1) &= \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)
\end{aligned} \tag{6}$$

where pos is the position index, i is the dimension index, and d_{model} is the dimension of the input. In this method, we use the category ID as the position index.

Learnable method This method is similar to the general embedding method, which is randomly initialized and trainable. Unlike the absolute position encoding, which has fixed values, this method has independent learnable parameters that can be integrated into the overall model for end-to-end learning.

Generative method This method converts the hierarchy tree into a heterogeneous graph with two types of nodes: category nodes and location nodes. Then, the GNN method is used to sum the embeddings (which are learned in Sect. 4.2.1) of

the location nodes under the same category and map them to the category embeddings. The learning process can be expressed as follows:

$$h_c = \sigma(\text{Sum}(h_p) \cdot W) \quad (7)$$

where h_p represents the embedding of the location nodes under the same category, W denotes the learnable parameter matrix, $\sigma(\cdot)$ indicates the activation function, and $h_c \in \mathbb{R}^{D_2}$ is the category embedding.

The generative method does not have independent learnable parameters such as the learnable method does. However, it can derive the category embeddings from the location embeddings, which are trainable, unlike the absolute position encoding method, which has fixed values.

4.2.3 Hierarchy information representation

In this study, we extract the top-to-bottom paths from the constructed hierarchy tree as the hierarchy information. To efficiently represent these paths, we combine the embeddings of all the nodes on the paths as follows:

$$h_h = h_c \parallel h_p \quad (8)$$

$$h_h = h_c + h_p \quad (9)$$

where h_h denotes the extracted hierarchy information and $\parallel$ is the vector concatenation operator.

Equation (9) requires that the category embedding and the location embedding have the same dimension, whereas Eq. (8) is more flexible. Moreover, the concatenation method makes it easier for the model to learn the category similarity between locations. Furthermore, the concatenation approach does not affect the encoding of the global location transition effects or the geospatial effects in the location embedding. Therefore, we adopt the concatenation method in this study. By doing so, the resulting hierarchy information not only includes the hierarchical relationship between categories and specific locations, but also preserves the transition effects and geospatial effects among locations.

4.3 Trajectory embedding generation

The generation of trajectory embeddings consists of two steps: first, the generation of spatial-temporal point embeddings, and second, the concatenation of multiple spatial-temporal points in chronological order into trajectory embeddings.

4.3.1 Spatial-temporal point embedding

The embedding of a spatial-temporal point integrates user and temporal embeddings, along with hierarchy information as detailed in Sect. 4.2.3.

User embedding For user embedding, we employ an embedding layer to map user IDs to dense vectors as the initial user embedding, denoted by $h_u \in \mathbb{R}^{D_3}$. This embedding can be learned continuously during model training.

Temporal embedding Aligning with findings from [39] that human mobility patterns demonstrate regularity across various time scales, including days and weeks, we encode time at these two scales. We first determine the day of the week (1–7) and then segment the 24-hr day into 48 half-hour timeslots, extracting the respective timeslot ID (1–48). Utilizing the positional encoding technique from Transformer, specified in Eq. (6), we generate temporal embeddings for both the day, $h_t^d \in \mathbb{R}^{D_4}$, and the week, $h_t^w \in \mathbb{R}^{D_4}$. The combined temporal embedding, $h_t \in \mathbb{R}^{D_4}$, is the sum of these two embeddings:

$$h_t = h_t^d + h_t^w \quad (10)$$

This approach enables precise modeling of the periodicity in user movement behavior on both day and week scales. Next, we concatenate h_u , h_h , and h_t together as the spatial-temporal point embedding, defined as:

$$h_{pe} = h_u \parallel h_h \parallel h_t \quad (11)$$

4.3.2 Trajectory embedding

A trajectory is a sequence of spatial-temporal points ordered by time. To obtain the embedding of a trajectory, we concatenate the embeddings of the spatial-temporal points in their temporal order. We denote this embedding as h_{tr} , which has a shape $[N, \sum_{i=1}^4 D_i]$. N is the number of spatial-temporal points in a trajectory. Different trajectories may have different values of N . For practical training, we set a fixed value of N and pad the trajectories that have fewer points than N with zeros. For the trajectories that have more points than N , we select the N closest points in time.

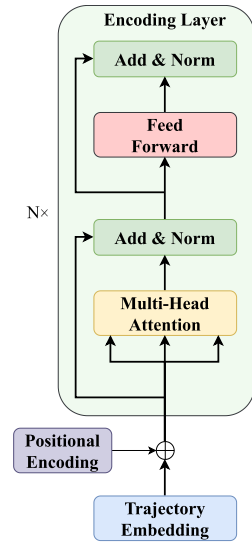
4.4 Encoder

We utilize the Transformer encoder architecture, renowned for its effectiveness in sequential tasks. The encoder features multiple layers, depicted in the light green box in Fig. 3, arranged sequentially. Each spatial-temporal point in a trajectory is initially encoded with position information through positional encoding (as defined in Eq. (6)) before being input into the first encoding layer. Each encoding layer comprises multi-head attention, a residual connection with layer normalization, and a feed-forward neural network (FFN).

4.4.1 Multi-head attention

Multi-head attention allows the encoder to focus on various aspects of the input sequences by dividing the query, key, and value vectors into multiple segments

Fig. 3 The encoder architecture of HAMTL, which is identical to that of the Transformer



and calculating attention separately for each. The computation of the multi-head attention mechanism is detailed as follows:

$$\text{head}_i = \text{softmax}\left(\frac{(h_{tr} \cdot W_{Q_i}) \cdot (h_{tr} \cdot W_{K_i})^T}{\sqrt{D}}\right)(h_{tr} \cdot W_{V_i}) \quad (12)$$

$$\text{MultiHead}(h_{tr}) = \text{head}_1 \parallel \text{head}_2 \cdots \parallel \text{head}_m$$

where m denotes the number of heads in the multi-head attention mechanism, $D = \sum_{i=1}^4 D_i$, and $W_{Q_i} \in \mathbb{R}^{D \times D/m}$, $W_{K_i} \in \mathbb{R}^{D \times D/m}$, and $W_{V_i} \in \mathbb{R}^{D \times D/m}$ are learnable transformation matrices. Equivalently, $W_Q = W_{Q_1} \parallel W_{Q_2} \cdots \parallel W_{Q_m}$, and W_K and W_V are defined similarly.

4.4.2 Residual connection and layer normalization

The encoding layer is composed of two sublayers: an attention sublayer, which includes Multi-Head Attention followed by Add & Norm, and an FNN sublayer, which comprises a Feed Forward network and Add & Norm. The Add & Norm component consists of residual connections and layer normalization. Residual connections help mitigate the vanishing gradient problem and maintain the flow of information, while layer normalization addresses internal covariate shift and facilitates faster convergence. The attention sublayer integrates these techniques as follows:

$$\text{AttSub}(h_{tr}) = \text{LN}(h_{tr} + \text{MultiHead}(h_{tr})) \quad (13)$$

where LN denotes the layer normalization.

4.4.3 Feed-forward neural network

FFN in the encoder is a pointwise nonlinear transformation that applies uniformly across all positions of the input and output representations. This design enables the encoder to discern complex interdependencies within these representations. FFN can be expressed as:

$$FFN(X) = ReLU(XW_1 + b_1)W_2 + b_2 \quad (14)$$

where $X = AttSub(h_{tr})$, and W_1, W_2, b_1 , and b_2 are learnable parameters. The output of the encoder is denoted by h_e , which has the same dimension as h_{tr} .

4.5 Decoder

Given that location categories have an inherent hierarchical relationship with specific locations, we propose a new hierarchical decoder architecture as shown in Fig. 4.

Prior hierarchy information In this study, we extracted the category-location transition probability matrix, denoted as $P \in \mathbb{R}^{N_c \times N_l}$, from the historical check-in data as the prior hierarchy information. N_c and N_l represent the number of categories and locations, respectively. We propose two methods to obtain P . The first method was formulated as

$$\omega_{c_i, l_j} = \frac{N_{l_j}}{\sum_{l_k \in c_i} N_{l_k}} \quad (15)$$

where c_i and l_j denote the i_{th} category and the j_{th} location under c_i , respectively, ω_{c_i, l_j} is the element in the corresponding position of P , and N_{l_j} is the number of check-ins in location l_j . This somewhat characterized the popularity of different locations within the same category.

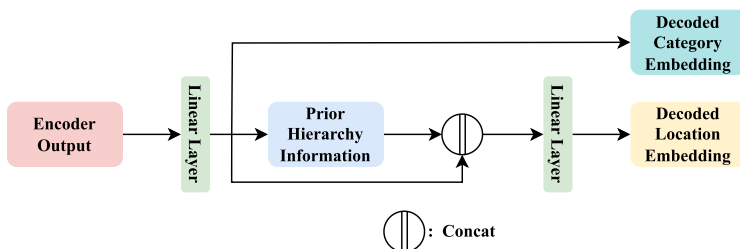


Fig. 4 Hierarchical Decoder. First, a linear layer is utilized to obtain the decoded category embedding for category prediction. Next, this decoded category embedding is multiplied by the prior hierarchy information matrix and concatenated with itself. Finally, the concatenated embeddings are processed through another linear layer, resulting in the decoded location embedding

The second approach was expressed as follows.

$$\omega_{c_i, l_j} = \frac{1}{N_{l_{c_i}}}, l_j \in c_i \quad (16)$$

where $N_{l_{c_i}}$ denotes the number of locations within category c_i . This approach treats locations within the same category equally.

The specific decoding process was as follows:

$$\begin{aligned} \tilde{Y}_c &= h_e \cdot W_1 + b_1 \\ \tilde{Y}_l &= \sigma((\tilde{Y}_c \cdot P) \parallel \tilde{Y}_c) \cdot W_2 + b_2 \end{aligned} \quad (17)$$

where $\tilde{Y}_c \in \mathbb{R}^{N \times N_c}$ and $\tilde{Y}_l \in \mathbb{R}^{N \times N_l}$ are the predicted category and location matrices, respectively, and $W_1 \in \mathbb{R}^{D \times N_c}$ and $W_2 \in \mathbb{R}^{(N_l + N_c) \times N_l}$ are the learnable parameter matrices.

The hierarchical decoder capitalizes on the inherent parent–child dependency between categories and locations. It first predicts the category for the next destination (e.g., restaurant). This category prediction inherently restricts the potential search space for the subsequent fine-grained location prediction (e.g., limiting candidate venues to hotspot restaurants). This process is formalized using the prior hierarchy matrix P , which encodes the conditional probability distribution of specific locations given their parent categories, derived from historical transition patterns. The operation $\tilde{Y}_c \cdot P$ adjusts the probabilities of locations based on their co-occurrence frequency with the predicted category—effectively down-weighting probabilities of locations incompatible with the category prediction (e.g., suppressing non-restaurant venues). Subsequently concatenating $\tilde{Y}_c$ preserves the category context information, enabling finer-grained refinement of location predictions within the category. This hierarchical constraint effectively mirrors human decision-making logic, where users typically first select an activity category before choosing a specific venue. This approach significantly reduces prediction ambiguity by leveraging structural dependencies within the data.

Algorithm 1 HAMTL

Input: Check-in history
Output: Model parameters: Θ

```

//Data Preparation
1: Split the dataset into training, validation, test sets in chronological order;
2: Extract trajectories from each set;
3: Construct the location graph  $\mathcal{G}$  from the training set;
4: Extract prior hierarchy information from the training set using Eq. 15 or 16.
//Training
5: Initialize model parameters:  $\Theta$ ;
6: Generate initial user embedding  $h_u$ , location embedding  $h_p$  and category embedding  $h_c$  using Embedding Layer, and time embeddings  $h_t^d$  and  $h_t^w$  using Positional Encoding;
7: for  $b = 1, 2, \dots, B$  do
    //e represents an epoch
8:   Load the training set;
9:   for  $b = 1, 2, \dots, B$  do
    //b represents a batch
10:    Update  $h_p$  on  $\mathcal{G}$  using Eqs. 2, 4 and 5;
11:    Generate  $h_c$  using Eq. 6 or Eq. 7 or Learnable Method;
12:    Extract hierarchy information using Eq. 8;
13:    Obtain spatial-temporal point embeddings using Eqs.10 and 11;
14:    Get trajectory embeddings;
15:    Encode trajectory embeddings using Eqs. 12 - 14;
16:    Decode the encoded trajectory embeddings using Eq.17;
17:    Calculate the losses using Eq.18;
18:    Update  $\Theta$  using Adam optimizer;
19:   end for
20:   Load the validation set and evaluate the predictions and losses;
21:   Adjust the learning rate or stop training based on validation performance;
22: end for
23: Return model parameters:  $\Theta$ .
```

4.6 Loss function

Our proposed method, HAMTL, addresses two multiclassification tasks: predicting locations and their categories. We use cross-entropy as the loss function for both tasks, assigning equal weight to each. The loss for the i_{th} trajectory of user u ($h_{S_u}^i = h_{p_1}, h_{p_2}, \dots, h_{p_N}$) is calculated as follows:

$$\begin{aligned}
\hat{Y}_l &= \text{softmax}(\tilde{Y}_l) \\
\hat{Y}_c &= \text{softmax}(\tilde{Y}_c) \\
\mathcal{L}_l &= - \sum_{i=2}^N y_{(l,p_i)} \log \hat{y}_{(l,p_i)} \\
\mathcal{L}_c &= - \sum_{i=2}^N y_{(c,p_i)} \log \hat{y}_{(c,p_i)} \\
\mathcal{L}_{S_u^i} &= \mathcal{L}_l + \mathcal{L}_c \\
\mathcal{L} &= \sum_{u \in U} \sum_{i=1}^m \mathcal{L}_{S_u^i}
\end{aligned} \tag{18}$$

where m is the number of trajectories of user u , which varies among users. During training, we use the first $N - 1$ spatial-temporal points of $h_{S_u^i}$ as input to obtain the predicted probabilities: $\hat{Y}_l \in \mathbb{R}^{N \times N_l}$ and $\hat{Y}_c \in \mathbb{R}^{N \times N_c}$. The cross-entropy is calculated using the last $N - 1$ spatial-temporal points as true values. The overall loss, $\mathcal{L}$, is the cumulative loss across all trajectories for all users.

We conduct end-to-end training of HAMTL, detailed in the pseudocode found in Algorithm 1.

5 Experiments, results, and discussions

We performed experiments using real-world datasets to answer the following questions:

- *RQ1*: How does HAMTL compare to existing baselines?
- *RQ2*: How do different methods of generating hierarchy information affect model performance?
- *RQ3*: How does prior hierarchy information influence model performance?
- *RQ4*: How does the hierarchical decoder architecture affect performance compared with the parallel architecture?

5.1 Datasets

We evaluated our proposed model using two real-world datasets,² Foursquare-TKY and Foursquare-NYC, as described in [44]. These datasets comprise user check-ins in Tokyo and New York City, respectively, spanning 10 months from April 12, 2012, to February 16, 2013, on the Foursquare platform. Each check-in records the user-ID, location-ID, category name, GPS coordinates, and timestamp. We refined each dataset by eliminating locations and users with fewer than

² <https://sites.google.com/site/yangdingqi/home/foursquare-dataset>.

10 check-ins and chronologically divided the data into training (80%), validation (10%), and test (10%) sets. From the training set, we constructed the location graph and extracted prior hierarchy information. We grouped consecutive check-ins into separate trajectories if they were more than 24 hr apart and removed trajectories consisting of only one check-in. The statistics of the two processed datasets are detailed in Table 1. For Foursquare-TKY, the distribution included 51,634 trajectories for training, 7087 for validation, and 6932 for testing. Similarly, for Foursquare-NYC, we prepared 18,781 trajectories for training, 2513 for validation, and 2417 for testing.

5.2 Experimental configuration

5.2.1 Comparative models

To evaluate the effectiveness of HAMTL, we compared it with seven state-of-the-art baselines.

ST-RNN [16] enhances traditional RNNs by adeptly encapsulating both temporal and spatial interdependencies among various locations, achieved through the incorporation of transition matrices that are cognizant of time and distance within each layer.

STGN [17] innovates within LSTM frameworks by embedding spatial-temporal gates, which precisely model the dynamics between consecutive check-ins.

TiSASRec [45] employs self-attention to simultaneously account for the absolute positions of items and the time intervals between them within a sequence.

STAN [8] unearth both the global spatial-temporal relationships among all check-ins, as well as the local correlations among recent check-ins by a dual self-attention mechanism.

GETNext [10] introduces a GNN module within Transformer, effectively capturing the transition of all users across various locations and individual user preferences.

DCRec [46] combines sequential pattern encoding with global collaborative relationship modeling, and it employs adaptive conformity-aware augmentation to address popularity bias.

MSTHN [47] utilizes a spatiotemporally enhanced GNN to capture and propagate spatial-temporal correlations effectively within user-POI interactions. Concurrently, it incorporates an interactive hypergraph neural network designed to efficiently capture complex high-order collaborative signals.

Table 1 Statistical information of the datasets

Dataset	User	Location	Category	Check-in	Trajectory
TKY	2282	7840	247	44,5734	65,653
NYC	1076	5081	209	14,6501	23,711

5.2.2 Performance metrics

To assess the performance of HAMTL and its comparative models, we implemented two evaluation metrics: top- k accuracy ($Acc@k$) and mean reciprocal rank (MRR). $Acc@k$ determines how often the actual next location appears within the top- k predictions for each trajectory. Conversely, MRR averages the reciprocal rankings of the correct next location across all predictions. The definitions for $Acc@k$ and MRR are given below:

$$Acc@k = \frac{1}{|T|} \sum_{i=1}^{|T|} \mathbb{I}(t(S_i) \in top_k(S_i)) \quad (19)$$

$$MRR = \frac{1}{|T|} \sum_{i=1}^{|T|} \frac{1}{rank(p(S_i))} \quad (20)$$

where $\mathbb{I}(\cdot)$ denotes the indicator function, T is the set of test trajectories, S_i is an individual trajectory within T , $top_k(S_i)$ identifies the top- k predicted locations for S_i , and $t(S_i)$ represents the true next location of S_i . We set k to 1, 5, 10, and 20. Higher scores on both metrics indicate superior model performance.

5.2.3 Implementation details

HAMTL was developed using PyTorch and DGL.³ All the experiments were run on a Linux server equipped with an Intel Xeon Gold 5218 CPU@2.30 GHz with 12 cores, a GeForce RTX 3090 GPU and 48 GB of RAM. For comparative models, we adopted the optimal parameters reported in the original publications. In HAMTL, the dimensions for user and location embeddings were set to 128, while category and temporal embeddings (for both day and week modes) were set at 16. For learning the location graph, we focused on optimizing $\mathbf{W} \in \mathbb{R}^{D \times K}$ and $\mathbf{q} \in \mathbb{R}^{K \times 1}$ within the semantic attention framework, setting K to 512 and using two graph convolution layers. The encoder configuration included two layers, dual attention heads, an FFN hidden dimension of 1024, and a dropout rate of 0.2. Optimization was carried out using the Adam optimizer with a learning rate of $1e-3$, a weight decay of $1e-3$, and a ReduceLROnPlateau strategy to adjust the learning rate based on the combined performance of $Acc@20$ and $Acc@1$. Trajectory length was maintained at 40, with batch sizes of 1500 for Foursquare-NYC and 1200 for Foursquare-TKY.

5.3 Performance comparison

We propose three methods for generating category embeddings (Sect. 4.2.2) and two methods for extracting prior hierarchical information [Eqs. (15), (16)]. These

³ <https://github.com/dmlc/dgl>.

methods give rise to six variants of HAMTL. Table 2 shows the notation and method for each variant.

Table 3 compares the performance of the HAMTL variants and the baseline methods on both datasets. As observed, while early RNN-based methods (e.g., ST-RNN and STGN) have made significant contributions to integrating spatial and temporal information, they struggle with the complexity and variability inherent in user mobility patterns. Recent advancements have been more successful in tackling these challenges. TiSASRec enhances performance by explicitly modeling time intervals between user interactions using a time-aware self-attention mechanism, which captures temporal dynamics more effectively, resulting in improved accuracy for user action prediction. Building on this, STAN employs a self-attention mechanism that dynamically adapts to both geographical and temporal data, further refining its ability to manage complex user interactions. This approach surpasses traditional RNN-based models by focusing more effectively on key aspects of user mobility.

DCRec combines sequential pattern encoding with global collaborative relationship modeling using adaptive conformity-aware augmentation. This allows it to effectively capture both transition patterns within sequences and dependencies between users across sequences. Compared with STAN, DCRec excels in modeling global collaborative relationships, enhancing prediction accuracy, although it sometimes overlooks other crucial spatial-temporal information. In contrast, MSTHN uses a spatial-temporal enhanced GNN to address this gap directly. It also integrates an interactive hypergraph network, which enhances its ability to capture complex, high-order collaborative signals, thereby achieving higher prediction accuracy than DCRec. Among the baseline models, GETNext stands out for effectively utilizing attention mechanisms along with global collaborative relationship modeling, significantly improving its ability to capture user transition behaviors while integrating interactions with other relevant spatial-temporal information.

All variants of HAMTL have demonstrated superior performance compared to GETNext, the state-of-the-art baseline, across all evaluated metrics. Notably,

Table 2 Variants of HAMTL

	Category embedding			Prior information	
	a	l	g	s	e
$HAMTL_{as}$	✓			✓	
$HAMTL_{ae}$	✓				✓
$HAMTL_{ls}$		✓		✓	
$HAMTL_{le}$		✓			✓
$HAMTL_{gs}$			✓	✓	
$HAMTL_{ge}$			✓		✓

The symbols a, l, and g denote the methods of category embedding generation based on the absolute position encoding method, the learnable method, and the generative method, respectively. The symbols s and e denote the methods of extracting the prior hierarchy information using Eqs. (15) and (16)

Table 3 Performance comparison between six variants of HAMTL and other baselines using the Foursquare-TKY and Foursquare-NYC datasets

	Foursquare-TKY				Foursquare-NYC					
	Acc@20	Acc@10	Acc@5	Acc@1	MRR	Acc@20	Acc@10	Acc@5	Acc@1	MRR
ST-RNN	0.4402	0.3436	0.2838	0.1366	0.2126	0.4463	0.3492	0.2887	0.1439	0.2148
STGN	0.4617	0.3788	0.3301	0.1632	0.2412	0.4998	0.4037	0.3321	0.1702	0.2564
TISASRec	0.4829	0.4148	0.3503	0.1712	0.2564	0.5503	0.4821	0.4176	0.1972	0.3082
STAN	0.4998	0.4462	0.3567	0.1742	0.2621	0.5796	0.5266	0.4537	0.2126	0.3221
DCRec	0.5123	0.4548	0.4023	0.2119	0.2832	0.5863	0.5382	0.4796	0.2296	0.3325
MSTHN	0.5515	0.4988	0.4198	0.2165	0.3082	0.6314	0.5867	0.4857	0.2338	0.3458
GETNext	0.5783	0.5122	0.4358	0.2246	0.3284	0.6621	0.6027	0.5047	0.2421	0.3598
HAMTL _{us}	0.6493	0.5841	0.4965	0.2638	0.3693	0.6671	0.6100	0.5269	0.2449	0.3687
HAMTL _{de}	0.6517	0.5818	0.4958	0.2625	0.3697	0.6667	0.6092	0.5284	0.2438	0.3674
HAMTL _{is}	0.6534	0.5845	0.4972	0.2592	0.3683	0.6677	0.6076	0.5248	0.2497	0.3707
HAMTL _{le}	0.6546	0.5869	0.4982	0.2657	0.3727	0.6681	0.6077	0.5252	0.2525	0.3721
HAMTL _{gs}	0.6449	0.5757	0.4899	0.2578	0.3638	0.6721	0.6154	0.5404	0.2517	0.3759
HAMTL _{ge}	0.6456	0.5746	0.4896	0.2602	0.3642	0.672	0.6145	0.5359	0.2543	0.3762

Bold and underline denote SOTA performance across all model variants

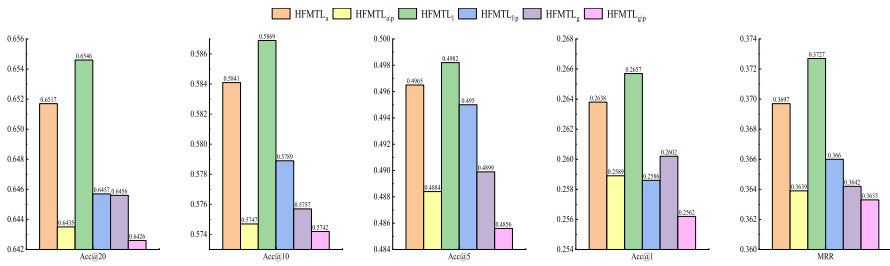


Fig. 5 Performance comparison of hierarchy information generation methods and prior hierarchy information on Foursquare-TKY. HAMTL_a denotes the absolute position encoding method and HAMTL_{a\p} omits the prior hierarchy information in the decoder of HAMTL_a. HAMTL_l denotes the learnable method and HAMTL_{l\p} omits the prior hierarchy information in the decoder of HAMTL_l. HAMTL_g denotes the generative method and HAMTL_{g\p} omits the prior hierarchy information in the decoder of HAMTL_g

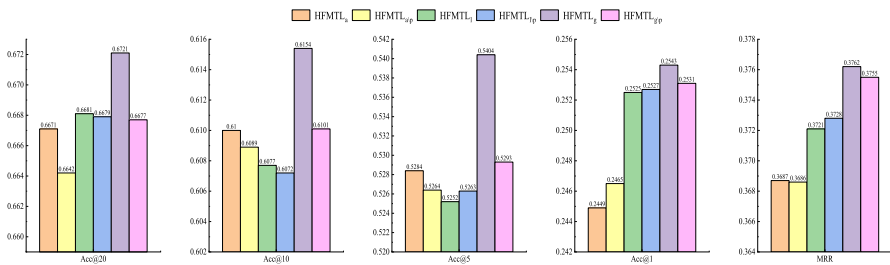


Fig. 6 Performance comparison of hierarchy information generation methods and prior hierarchy information on Foursquare-NYC. HAMTL_a denotes the absolute position encoding method and HAMTL_{a\p} omits the prior hierarchy information in the decoder of HAMTL_a. HAMTL_l denotes the learnable method and HAMTL_{l\p} omits the prior hierarchy information in the decoder of HAMTL_l. HAMTL_g denotes the generative method and HAMTL_{g\p} omits the prior hierarchy information in the decoder of HAMTL_g

HAMTL_{le} shows a remarkable increase of over 18% in the *Acc@1* metric for Foursquare-TKY, while HAMTL_{gs} improves *Acc@5* by more than 7% for Foursquare-NYC. These improvements suggest that the integration of hierarchy information and the application of multi-task learning strategies significantly boost the predictive accuracy of the model. These findings effectively address **RQ1**.

5.4 Comparison of methods for hierarchy information generation

In this study, hierarchy information generation refers to the category embedding generation. We introduce three approaches: the absolute position encoding method, the learnable method, and the generative method, to generate category embeddings. Furthermore, we propose two methods for extracting prior hierarchy information. Consequently, we select the best result for each evaluation metric obtained from the two methods to compare the performance of the above three approaches for category

embedding generation. The performance comparison is shown in Figs. 5 and 6 (HAMTL_a, HAMTL_l, and HAMTL_g).

Compared with the other two methods, the generative method, HAMTL_g, outperforms the other two methods on Foursquare-NYC, but underperforms on Foursquare-TKY. The generative method relies on learning for location embeddings and does not have independent learnable parameters like the learnable method does. This method can effectively model and train the hierarchical relationships between categories and locations when the training data are limited. However, when sufficient training data exist, it fails to fulfill the modeling requirements compared with the other two methods of independent modeling categories. Foursquare-TKY has approximately the same number of categories as Foursquare-NYC, but three times more data to train the model. Therefore, the absolute position encoding method and the learnable method (HAMTL_a and HAMTL_l), which model categories independently, outperform the generative method on Foursquare-TKY, but not on Foursquare-NYC.

When comparing the learnable method and the absolute position encoding method, both of which model categories independently, it becomes clear that the learnable method, HAMTL_l, does not consistently outperform the absolute position encoding method, HAMTL_a, when the data are limited. For example, on Foursquare-NYC, HAMTL_l performs worse than HAMTL_a with respect to $Acc@10$ and $Acc@5$. However, when the data are abundant, the learnable method consistently outperforms the absolute position encoding method across all the metrics. For example, on Foursquare-TKY, HAMTL_l outperforms HAMTL_a across all the metrics. These results provide an answer to RQ2.

5.5 Exploring prior hierarchy information

In this study, we investigate the impact of prior hierarchy information on the model and compare the two methods [Eqs. (15), (16)] for extracting this information. We denote the model without the prior hierarchy information module in Fig. 4 as HAMTL_{*∖p}. For example, HAMTL_{a∖p} represents HAMTL_{as} or HAMTL_{ae} without prior hierarchy information. We still select the best result for each evaluation metric from the two methods and compare them with HAMTL_{*∖p}. The comparison results are presented in Figs. 5 and 6.

For all the metrics, the variants with prior hierarchy information exhibit superior performance compared with those without prior hierarchy information on Foursquare-TKY. For Foursquare-NYC, HAMTL_g outperforms HAMTL_{g∖p} across all the metrics. However, on certain metrics, HAMTL_a and HAMTL_l perform worse than HAMTL_{a∖p} and HAMTL_{l∖p}. For example, HAMTL_a is no better than HAMTL_{a∖p} in the metric of $Acc@1$. The results suggest that incorporating prior hierarchy information improves the overall model performance, and this improvement is due to the effective modeling of hierarchy information. Moreover, when comparing HAMTL_{*s} with HAMTL_{*e} on Foursquare-TKY and Foursquare-NYC (see Table 3), we observe no clear advantages or disadvantages for either method. These findings answer RQ3.

Table 4 Performance comparison of hierarchical and parallel decoder architectures on datasets Foursquare-TKY and Foursquare-NYC

	Foursquare-TKY					Foursquare-NYC				
	Acc@20	Acc@10	Acc@5	Acc@1	MRR	Acc@20	Acc@10	Acc@5	Acc@1	MRR
$HAMTL_u$	<u>0.6517</u>	<u>0.5841</u>	<u>0.4965</u>	<u>0.2638</u>	<u>0.3697</u>	<u>0.6671</u>	<u>0.6100</u>	<u>0.5284</u>	0.2449	0.3687
$HAMTL_{u,pd}$	0.6497	0.5802	0.4943	0.2611	0.3683	0.6611	0.6057	0.5227	<u>0.2477</u>	<u>0.3691</u>
$HAMTL_l$	<u>0.6546</u>	<u>0.5869</u>	<u>0.4982</u>	<u>0.2657</u>	<u>0.3727</u>	<u>0.6681</u>	<u>0.6077</u>	0.5252	0.2525	0.3721
$HAMTL_{l,pd}$	0.6535	0.5836	0.4956	0.2652	0.3701	0.662	0.6068	0.5264	<u>0.2532</u>	<u>0.3728</u>
$HAMTL_g$	0.6456	0.5757	0.4899	0.2602	0.3642	<u>0.6721</u>	<u>0.6154</u>	<u>0.5404</u>	<u>0.2543</u>	<u>0.3762</u>
$HAMTL_{g,pd}$	<u>0.6485</u>	<u>0.5808</u>	<u>0.4902</u>	<u>0.2607</u>	<u>0.3667</u>	0.6586	0.6002	0.5226	0.2478	0.3686

Underline indicates ablation comparisons within the same model variant (3 variants total), ensuring consistent emphasis hierarchy without overlapping with SOTA indicators

$HAMTL_u$, $HAMTL_l$, and $HAMTL_g$, with the same implications as in Figs. 5 and 6, are the hierarchical architecture. $HAMTL_{u,pd}$, $HAMTL_{l,pd}$, and $HAMTL_{g,pd}$ are the parallel architecture

5.6 Comparison of hierarchical and parallel decoder architectures

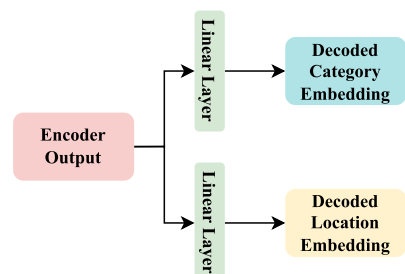
In this study, we select the best result for each evaluation metric from the two methods of extracting the prior hierarchy information in the hierarchical decoder and compare them with those of the parallel decoder (shown in Fig. 7) denoted as $\text{HAMTL}_{*,pd}$. The results of the comparison can be found in Table 4.

For Foursquare-TKY, HAMTL_a and HAMTL_l outperform $\text{HAMTL}_{a,pd}$ and $\text{HAMTL}_{l,pd}$ across all the metrics. However, HAMTL_g performs worse than $\text{HAMTL}_{g,pd}$ across all the metrics. For Foursquare-NYC, HAMTL_g performs better than $\text{HAMTL}_{g,pd}$ across all the metrics. HAMTL_a and HAMTL_l outperform $\text{HAMTL}_{a,pd}$ and $\text{HAMTL}_{l,pd}$ in certain metrics. Combining the findings in Sect. 5.4, we can infer that when modeling hierarchy information is relatively more effective, the hierarchical architecture brings more benefits. Conversely, the parallel architecture is more advantageous. In other words, the superiority of the hierarchical architecture is based on the effective modeling of hierarchy information. These are responses to *RQ4*.

6 Conclusion

In this study, we propose HAMTL, a novel hierarchy aware-based multi-task learning framework designed for user location prediction. HAMTL consists of four main components: hierarchy information extraction, trajectory embedding generation, Transformer-based encoder, and a hierarchical decoder. The hierarchy information extraction module introduces three methods (the absolute position encoding method, learnable method, and generative method) to obtain high-level hierarchy information, specifically category embeddings. In addition, we propose two methods to extract prior hierarchy information in the decoder. Through extensive experiments, we demonstrate that HAMTL significantly outperforms state-of-the-art baselines, achieving up to 18% Acc@1 improvement. Furthermore, the hierarchical decoder architecture is beneficial for HAMTL because of the effective modeling of hierarchy information. Our approach presents two main limitations. First, as the category-location transition matrix is solely derived from historical statistics, it inherently lacks trainable parameters required for adaptive optimization. Second, the generalization capability of the same category

Fig. 7 Parallel decoder. Two linear layers are used to obtain the decoded category embedding and the decoded location embedding, respectively



embedding generation method is constrained across datasets of varying scales. For future work, we intend to improve the prediction performance by modeling user similarity on the basis of their behavioral trajectories. In addition, we aim to investigate the impact of incorporating more levels of hierarchy information on the prediction performance.

Acknowledgements The authors gratefully appreciate anonymous reviewers for their valuable comments.

Author Contributions YW contributed to the methodology, experiments and the main manuscript writing. HC contributed to the formal analysis and resource. SL contributed to the supervision and the manuscript review and revision. JZ contributed to the manuscript review and revision. LW and XC contributed to the validation. YH contributed to the supervision.

Funding This work is supported by the Major Scientific and Technological Special Project of Henan Province (221100210100), Science and Technology Innovation leading Talents Subsidy Project of Central Plains (244200510038), and Henan Provincial Science and Technology Research Project (252102210038).

Data Availability The link to the original datasets is <https://sites.google.com/site/yangdingqi/home/foursquare-dataset>.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Qiu S, Pi D, Wang Y, Liu Y (2023) Novel trajectory privacy protection method against prediction attacks. *Expert Syst Appl* 213:118870. <https://doi.org/10.1016/j.eswa.2022.118870>
2. Byun H, Choi Y, Kim C-K (2023) Aspect-oriented unsupervised social link inference on user trajectory data. *Inf Sci* 626:249–261. <https://doi.org/10.1016/j.ins.2023.01.022>
3. Jin G, Liang Y, Fang Y, Shao Z, Huang J, Zhang J, Zheng Y (2024) Spatio-temporal graph neural networks for predictive learning in urban computing: a survey. *IEEE Trans Knowl Data Eng* 36(10):5388–5408. <https://doi.org/10.1109/TKDE.2023.3333824>
4. Zhong L, Zeng J, Wang Z, Zhou W, Wen J (2024) Scfl: spatio-temporal consistency federated learning for next poi recommendation. *Inf Process Manag* 61(6):103852. <https://doi.org/10.1016/j.ipm.2024.103852>
5. Hakyemez TC, Badur B (2023) Incorporating park events into crime hotspot prediction on street networks: a spatiotemporal graph learning approach. *Appl Soft Comput* 148:110886. <https://doi.org/10.1016/j.asoc.2023.110886>
6. Yang D, Fankhauser B, Rosso P, Cudre-Mauroux P (2020) Location prediction over sparse user mobility traces using RNNs. In: *Proceedings of the 29th International Joint Conference on Artificial Intelligence*, pp 2184–2190. <https://doi.org/10.24963/ijcai.2020/302>
7. Sun K, Qian T, Chen T, Liang Y, Nguyen QVH, Yin H (2020) Where to go next: Modeling long- and short-term user preferences for point-of-interest recommendation. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol 34, pp 214–221. <https://doi.org/10.1609/aaai.v34i01.5353>
8. Luo Y, Liu Q, Liu Z (2021) Stan: spatio-temporal attention network for next location recommendation. In: *Proceedings of the Web Conference 2021*, pp 2177–2185. <https://doi.org/10.1145/3442381.3449998>
9. Lin Y, Wan H, Guo S, Lin Y (2021) Pre-training context and time aware location embeddings from spatial-temporal trajectories for user next location prediction. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol 35, pp 4241–4248. <https://doi.org/10.1609/aaai.v35i5.16548>

10. Yang S, Liu J, Zhao K (2022) Getnext: trajectory flow map enhanced transformer for next poi recommendation. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 1144–1153. <https://doi.org/10.1145/3477495.3531983>
11. Cao G, Cui S, Joe I (2023) Improving the spatial-temporal aware attention network with dynamic trajectory graph learning for next point-of-interest recommendation. *Inf Process Manag* 60(3):103335. <https://doi.org/10.1016/j.ipm.2023.103335>
12. Liu Z, Meng L, Sheng QZ, Chu D, Yu J, Song X (2024) Poi recommendation for random groups based on cooperative graph neural networks. *Inf Process Manag* 61(3):103676. <https://doi.org/10.1016/j.ipm.2024.103676>
13. Wang Y, Chen H, Liu S, Wang K, Li X, Hu Y (2024) Iemtlf: interaction-enhanced multi-task learning framework for next location prediction. *Inf Sci* 661:120153. <https://doi.org/10.1016/j.ins.2024.120153>
14. Zeng J, Tao H, Tang H, Wen J, Gao M (2025) Global and local hypergraph learning method with semantic enhancement for poi recommendation. *Inf Process Manag* 62(1):103868. <https://doi.org/10.1016/j.ipm.2024.103868>
15. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: Proceedings of the 31st international conference on neural information processing systems, vol 30. Long Beach, California, USA. Curran Associates Inc., Red Hook, NY, USA. <https://doi.org/10.5555/3295222.3295349>
16. Liu Q, Wu S, Wang L, Tan T (2016) Predicting the next location: a recurrent model with spatial and temporal contexts. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 30. <https://doi.org/10.1609/aaai.v30i1.9971>
17. Zhao P, Luo A, Liu Y, Zhuang F, Xu J, Li Z, Sheng VS, Zhou X (2020) Where to go next: a spatio-temporal gated network for next poi recommendation. *IEEE Trans Knowl Data Eng* 34(5):2512–2524. <https://doi.org/10.1109/TKDE.2020.3007194>
18. Liu CH, Wang Y, Piao C, Dai Z, Yuan Y, Wang G, Wu D (2020) Time-aware location prediction by convolutional area-of-interest modeling and memory-augmented attentive LSTM. *IEEE Trans Knowl Data Eng* 34(5):2472–2484. <https://doi.org/10.1109/TKDE.2020.3005735>
19. Kipf TN, Welling M (2016) Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*
20. He X, Deng K, Wang X, Li Y, Zhang Y, Wang M (2020) Lightgcn: Simplifying and powering graph convolution network for recommendation. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 639–648. <https://doi.org/10.1145/3397271.3401063>
21. Hu X, Chen H, Liu S, Jiang H, Chu G, Li R (2022) Btg: a bridge to graph machine learning in telecommunications fraud detection. *Futur Gener Comput Syst* 137:274–287. <https://doi.org/10.1016/j.future.2022.07.020>
22. Liu Y, Wu H, Rezaee K, Khosravi MR, Khalaf OI, Khan AA, Ramesh D, Qi L (2022) Interaction-enhanced and time-aware graph convolutional network for successive point-of-interest recommendation in travelling enterprises. *IEEE Trans Industr Inf* 19(1):635–643. <https://doi.org/10.1109/TII.2022.3200067>
23. Qin Y, Wang Y, Sun F, Ju W, Hou X, Wang Z, Cheng J, Lei J, Zhang M (2023) Disenpoi: disentangling sequential and geographical influence for point-of-interest recommendation. In: Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining, pp 508–516. <https://doi.org/10.1145/3539597.3570408>
24. Long J, Chen T, Nguyen QVH, Yin H (2023) Decentralized collaborative learning framework for next poi recommendation. *ACM Trans Inf Syst* 41(3):66. <https://doi.org/10.1145/3555374>
25. Yin F, Liu Y, Shen Z, Chen L, Shang S, Han P (2023) Next poi recommendation with dynamic graph and explicit dependency. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 37, pp 4827–4834. <https://doi.org/10.1609/aaai.v37i4.25608>
26. Kong X, Chen Z, Li J, Bi J, Shen G (2024) Kgnext: knowledge-graph-enhanced transformer for next poi recommendation with uncertain check-ins. *IEEE Trans Comput Soc Syst* 11(5):6637–6648. <https://doi.org/10.1109/TCSS.2024.3396506>
27. Lai Y, Su Y, Wei L, He T, Wang H, Chen G, Zha D, Liu Q, Wang X (2024) Disentangled contrastive hypergraph learning for next poi recommendation. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 1452–1462. <https://doi.org/10.1145/3626772.3657726>
28. Zhang Y, Yang Q (2021) A survey on multi-task learning. *IEEE Trans Knowl Data Eng* 34(12):5586–5609. <https://doi.org/10.1109/TKDE.2021.3070203>

29. Zhao Y, Wang X, Che T, Bao G, Li S (2023) Multi-task deep learning for medical image computing and analysis: a review. *Comput Biol Med* 153:106496. <https://doi.org/10.1016/j.combiomed.2022.106496>
30. Ghosh S, Ekbal A, Bhattacharyya P (2023) Vad-assisted multitask transformer framework for emotion recognition and intensity prediction on suicide notes. *Inf Process Manag* 60(2):103234. <https://doi.org/10.1016/j.ipm.2022.103234>
31. Zou W, Zhang W, Wu W, Tian Z (2024) A multi-task shared cascade learning for aspect sentiment triplet extraction using Bert-MRC. *Cogn Comput* 16(4):1554–1571. <https://doi.org/10.1007/s12559-024-10247-7>
32. Sobhaninia Z, Karimi N, Khadivi P, Samavi S (2023) Brain tumor segmentation by cascaded multiscale multitask learning framework based on feature aggregation. *Biomed Signal Process Control* 85:104834. <https://doi.org/10.1016/j.bspc.2023.104834>
33. Rahman MM, Marculescu R (2024) G-cascade: Efficient cascaded graph convolutional decoding for 2d medical image segmentation. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp 7728–7737
34. Jin J, Chen X, Zhang W, Chen Y, Jiang Z, Zhu Z, Su Z, Yu Y (2022) Multi-scale user behavior network for entire space multi-task learning. In: *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pp 874–883. <https://doi.org/10.1145/3511808.3557405>
35. Cheng Z, Han S, Liu F, Zhu L, Gao Z, Peng Y (2023) Multi-behavior recommendation with cascading graph convolution networks. In: *Proceedings of the ACM Web Conference 2023*, pp 1181–1189. <https://doi.org/10.1145/3543507.3583439>
36. Yan M, Cheng Z, Gao C, Sun J, Liu F, Sun F, Li H (2023) Cascading residual graph convolutional network for multi-behavior recommendation. *ACM Trans Inf Syst* 42(1):10. <https://doi.org/10.1145/3587693>
37. Kumar KN, Essa I, Ha S (2023) Cascaded compositional residual learning for complex interactive behaviors. *IEEE Robot Autom Lett* 8(8):4601–4608. <https://doi.org/10.1109/LRA.2023.3286171>
38. Jiang S, Zhao C (2024) A cascaded embedding method with graph neural network for multi-behavior recommendation. *Int J Mach Learn Cybern* 15(6):2513–2526. <https://doi.org/10.1007/s13042-023-02045-8>
39. Gonzalez MC, Hidalgo CA, Barabasi A-L (2008) Understanding individual human mobility patterns. *Nature* 453(7196):779–782. <https://doi.org/10.1038/nature06958>
40. Tobler WR (1970) A computer movie simulating urban growth in the Detroit region. *Econ Geogr* 46(sup1):234–240
41. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y (2017) Graph attention networks. *arXiv preprint arXiv:1710.10903*
42. Chen M, Wei Z, Huang Z, Ding B, Li Y (2020) Simple and deep graph convolutional networks. In: *International Conference on Machine Learning*, pp 1725–1735
43. Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*
44. Yang D, Zhang D, Zheng VW, Yu Z (2014) Modeling user activity preference by leveraging user spatial temporal characteristics in lbsns. *IEEE Trans Syst Man Cybern Syst* 45(1):129–142. <https://doi.org/10.1109/TSMC.2014.2327053>
45. Li J, Wang Y, McAuley J (2020) Time interval aware self-attention for sequential recommendation. In: *Proceedings of the 13th International Conference on Web Search and Data Mining*, pp 322–330. <https://doi.org/10.1145/3336191.3371786>
46. Yang Y, Huang C, Xia L, Huang C, Luo D, Lin K (2023) Debaised contrastive learning for sequential recommendation. In: *Proceedings of the ACM Web Conference 2023*, pp 1063–1073. <https://doi.org/10.1145/3543507.3583361>
47. Lai Y, Su Y, Wei L, Chen G, Wang T, Zha D (2023) Multi-view spatial-temporal enhanced hypergraph network for next poi recommendation. In: *International Conference on Database Systems for Advanced Applications*, pp 237–252. https://doi.org/10.1007/978-3-031-30672-3_16

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Yahui Wang^{1,2} · Hongchang Chen² · Shuxin Liu² · Junjie Zhang² · Lan Wu² · Xiaoyan Cui³ · Yuxiang Hu²

✉ Shuxin Liu
liushuxin11@126.com

Yahui Wang
wangyahuidl@ncwu.edu.cn

Hongchang Chen
ndscchc@139.com

Junjie Zhang
jj961004@stu.xjtu.edu.cn

Lan Wu
lanwundsc@163.com

Xiaoyan Cui
992086703@qq.com

Yuxiang Hu
chxachxa@126.com

¹ North China University of Water Resources and Electric Power, Zhengzhou 450045, Henan, China

² National Digital Switching System Engineering and Technological Research Center, Zhengzhou 450002, Henan, China

³ 95897 Troops of the PLA, Zhengzhou, China